League Of Legends Roles Analysis

Alessandro Relato

2025-01-13

League of Legends Roles Analysis

For this project I decided to create and analyze a graph about how roles of League of Legends game interact between them.

Introduction

League of Legends is a videogame for PC created by Riot Games, in which 2 teams of 5 players combat against each other aiming to destroy the enemy base, called nexus. Every player has a role (Toplane, Jungler, Midlane, ADCarry or Support) depending on their duty and on which part of the map they are playing. The Toplaner plays in the upper lane and they are usually left alone because they play champions that can be usefull even if not helped and because this side of the map has less important things. The Jungler farms neutral monsters that are in the space between other lanes. They sometimes help other players to get a kill against their laner or to put pressure to gain advantage. The Midlaner stays in the lane that covers the diagonal of the map. They usually play champion that needs to get a lot of resources in the initial part of the game or champions that are more powerful as the game goes on. They need help to get resources or to survive, on the other side the lane is the center of the map so it is important also to control it for controlling the entire map. The ADCarry usually plays champions that alone are really really weak and that needs a lot of resources to become strong. They play in the same lane of the support and are usually helped also by the Jungler because in this side of the map there are useful neutral epic monsters. In the end we have the Support thaty can play mainly two types of champions: enchanters and engagers. First ones give healings and shields to allies, instead engagers have tools to start plays forcing enemies to fight.

With this study I aim to analyze if the in-game perception that Toplaners are basically ignored by the most of the players. Moreover I want to discover which role is the most interactive. Interactions are measured in common participation to a champion kill because of how informations are given by the API. Like the introduction to the game suggests, I am expecting that the Jungler is the most interactive role really closed to the support and the Toplane is the last one.

Creation of the graph

I created the graph by myself (with the help of chatgpt to learn how API calls worked). To do so I called the Riot API asking the Ids of the best 300 players in the EUW server. Then I asked the Ids of twenty of the last matches for every player. Using the match Ids I got the infos (which players played, who won, and so on) and the timelines (a description of the status of the game made every sixty seconds) for every match. All these calls were made using the *Request_Riot_Api.py* script.

Once I got the informations I used another Python script, *Graph_creator.py* to extract from the json I got from API calls only the infos I needed to create the graph. This second script for every match extracts the players' ids that was part of that match and puts them in the csv of the nodes. Then it searches in the

timeline files every Kill interaction and saves all the interactions in csv of the edges (counting the number of times two players interact).

Graph analysis

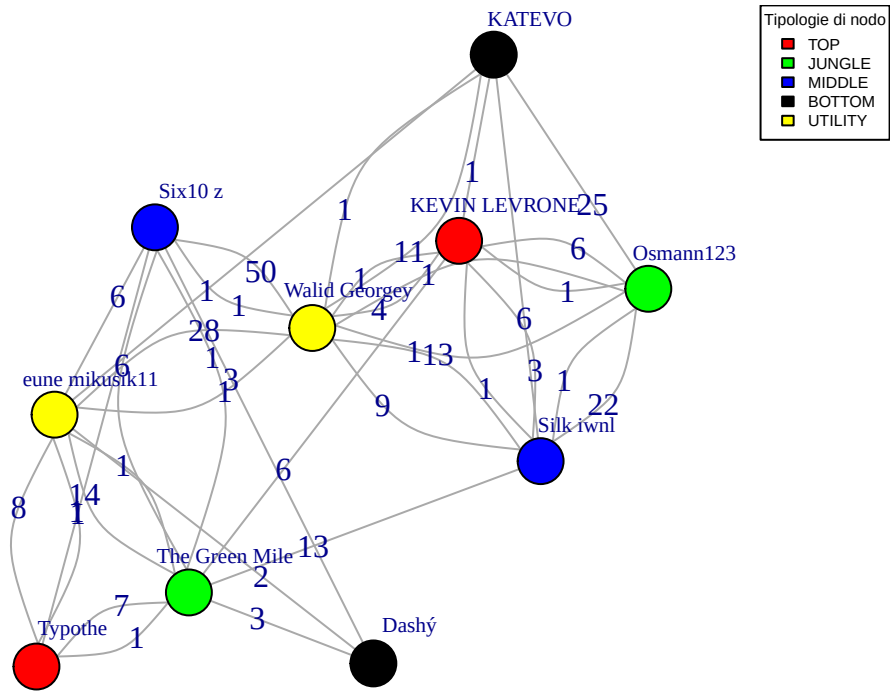
Initialization

Firstly I import the graph and all the libraries that will be needed. Then I assign a color attribute to the nodes so that I will have different roles colored in different colors. Lastly I am going to display a sub graph with the nodes of the first game that is in my data using it as an example of how my graph is.

```
nodes <- read.csv("nodes.csv", stringsAsFactors = FALSE, fileEncoding = "UTF-8")
edges <- read.csv("edges.csv", stringsAsFactors = FALSE, fileEncoding = "UTF-8")

g <- graph_from_data_frame(d=edges, vertices = nodes, directed = FALSE)
colors <- c("Red", "Green", "Blue", "Black", "Yellow")
node_type <- c("TOP", "JUNGLE", "MIDDLE", "BOTTOM", "UTILITY")
names(colors) <- node_type
V(g)$color <- colors[V(g)$role]

par(mar=c(3, 3, 1, 1))
# Plotting a subgraph because the entire graph is too big and
# it would be a complete mess if diplayed all
sub_g <- subgraph(g, vids=V(g)[1:10])
plot(sub_g, vertex.color=V(sub_g)$color, vertex.label=V(sub_g)$riotId,
     layout=layout_with_dh, vertex.size=15, vertex.label.cex=0.7,
     vertex.label.dist=2.3, edge.label=E(sub_g)$weight, edges.label.cex=0.3)
legend("topright", legend=names(colors), fill=colors, title="Tipologie di nodo", cex=0.5)
```




```
par(mar=c(5, 4, 4, 2))
```

Centrality studies

Firstly I study the degree centrality of the graph and then I group the results by role. I expect that, as I said in the introduction, the mean degree centrality of Junglers and Supports will be the higher and Toplaner will have the lowest one.

```
dc <- strength(g)
names(dc) <- V(g)$role
dc_top <- dc["TOP"]
dc_jungle <- dc["JUNGLE"]
dc_mid <- dc["MIDDLE"]
dc_bot <- dc["BOTTOM"]
dc_supp <- dc["UTILITY"]
results <- c(TOP=mean(dc_top), JUNGLE=mean(dc_jungle), MIDLANE=mean(dc_mid),
             BOTTOM=mean(dc_bot),SUPPORT=mean(dc_supp))
results
```

```
##     TOP  JUNGLE MIDLANE  BOTTOM SUPPORT
##     310    2486    2436     176    4615
```

The first analysis reveals that my prediction was wrong and the highest degree centrality is the one of Supports with almost the double of Junglers and Midlaners. This makes me think that Supports in Challenger

rank (the best 300 players) play a lot more around the map than lower ranks. On the other hand we can see that on average Toplaners have more interactions than ADCarries even if ADCarries and Supports play almost half of the game in the same lane.

After seeing the means, I would like to see the absolute values: which role has the central node using degree centrality? And which role has the lowest in the standing?

```
best_five <- sort(dc, decreasing = TRUE)[1:5]
worst_five <- sort(dc, decreasing = TRUE)[(length(dc)-4):length(dc)]
best_five
```

```
## BOTTOM JUNGLE    TOP JUNGLE MIDDLE
##   9570   9057   7928   7321   7158
```

```
worst_five
```

```
##    TOP MIDDLE    TOP    TOP    TOP
##      0      0      0      0      0
```

The highest node is an ADCarry role. This can be explained by the fact that this role prefers to play grouped with a lot of allies instead of playing alone because it needs some help to work properly. Moreover in the last part of games when there is a higher chance to make kills.

Nodes with 0 degree wasn't something that I expected because of how I created the data. But seeing them makes me think about the small amount of games that have to be remaked due to some afk players. Probably these nodes are players that aren't in my 300 players' pool because they have been matched in a game in which there was one of my 300 and the game ended with a remake. Remake happens when one player in game is recognized as afk. The system takes 90 seconds to do it, so it is near impossible for player to have some interaction in that short period of time

Now I would like to analyse how much roles interact between them. To do so I will filter the graph two roles at a time and will calculate the mean degree centrality in the filtered graph. All the results will be put in a matrix. I am expecting that Toplaners have the highest centrality with Junglers (and maybe with Supports seeing the previous results), Junglers and Supports similar centrality with everyone (maybe Junglers have higher centrality with Toplaners, and Supports higher with ADCarries). I think Midlaners will have a higher centrality with Junglers and Supports than with ADCarries and Toplaners. In the end ADCarries could have a similar centrality with Junglers and Midlaners, little higher with Supports and significantly lower with Toplaners.

```
roles <- c("TOP", "JUNGLE", "MIDDLE", "BOTTOM", "UTILITY")
result <- matrix(NA, nrow=5, ncol=5)
i=1
while(i<6) {
  j=1
  while(j<6) {
    if(i!=j) {
      selected_nodes <- V(g)[role %in% c(roles[i], roles[j])]
      subgraph <- subgraph(g, vids=selected_nodes)
      result[i, j] <- mean(degree(subgraph))
      result[j, i] <- result[i, j]
    }
    j=j+1
  }
  i=i+1
```

```r
}
result <- cbind(result, rowMeans(result, na.rm = TRUE))
result <- rbind(result, colMeans(result, na.rm = TRUE))
colnames(result) <- c("TOP", "JUNGLE", "MID", "BOT", "SUPPORT", "MEAN")
rownames(result) <- c("TOP", "JUNGLE", "MID", "BOT", "SUPPORT", "MEAN")
result <- round(result, 2)
result
```

```
##           TOP JUNGLE   MID   BOT SUPPORT  MEAN
## TOP        NA  33.40 28.41 25.18   23.40 27.60
## JUNGLE  33.40     NA 27.34 24.04   21.91 26.67
## MID     28.41  27.34    NA 20.46   18.62 23.71
## BOT     25.18  24.04 20.46    NA   16.60 21.57
## SUPPORT 23.40  21.91 18.62 16.60      NA 20.13
## MEAN    27.60  26.67 23.71 21.57   20.13 23.93
```

Surprisingly Toplaners have the highest centrality with every role. And that is confirmed also by the means. The second role is the Jungle which was something that I was expecting. Seeing the previous results I surely didn't expect that Support would have been the lowest one. After a lot of thinking I come to a conclusion: probably the reason of this result is that Toplaners tend to play champions that have more area damaging abilities, so in a fight they take a lot of assists. On the other hand Supports, specially in this status of the balance of League of Legends, play enchanters (champions with healing and shielding abilities) to save their ADCarry. In a fight Supports focus on ADCarries and don't shield many other players getting less assists. Moreover Supports are usually one of the easiest role to kill because they have less resources than everyone, thus it is common to kill Supports first. If they are the firsts to die, for sure can't contribute to the fight and can't take assists.

About other centralities I think that aren't so meaningful in this context because analyse the graph in his entirety and that's something that I don't need. For example the betweenness centrality in this graph would point out in how many games that player is among the total amount of games in the data set.

Clustering analysis

With clustering analysis I want to discover if players of the same role behave similar. So I will compute the hierarchical clustering and then I will cut the tree at five clusters. For every cluster I will calculate the % of every role in the cluster. I a expecting that this won't be a perfect division of roles, but will be close to that.

```r
A <- as_adjacency_matrix(g, sparse=FALSE)
# Eliminating the isolated nodes
A_filtered <- A[(rowSums(A)!=0), (colSums(A)!=0)]

# Cosine similarity
# euclidean <- function(x) {sqrt(x %*% x)}
# d <- apply(A_filtered, 2, euclidean)
# D <- diag(1/d)
# S <- D %*% t(A_filtered) %*% A_filtered %*% D
# D <- 1-S
# d <- as.dist(D)

# Pearson similarity
S <- cor(A_filtered)
D <- 1-S
d <- as.dist(D)
```

```r
# Average-linkage clustering method
clustering <- hclust(d, method = "average")
# Cutting at 5 clusters
clusters <- cutree(clustering, k=5)

clusters_data <- data.frame(id=V(delete_vertices(g,
                                                 V(g)[degree(g)==0]))$name,
                                                 cluster=clusters)
combined_data <- merge(clusters_data, nodes, by="id")

role_distribution <- combined_data %>%
  group_by(cluster, role) %>%
  summarise(count=n(), .groups="drop") %>%
  ungroup()
role_distribution <- role_distribution %>%
  group_by(cluster) %>%
  mutate(percent=round(count/sum(count)*100, 2)) %>%
  mutate(total=sum(count)) %>%
  ungroup()
role_distribution
```

```
## # A tibble: 24 x 5
##    cluster role    count percent total
##      <int> <chr>   <int>   <dbl> <int>
##  1       1 BOTTOM    226    8.09  2793
##  2       1 JUNGLE    692   24.8   2793
##  3       1 MIDDLE    386   13.8   2793
##  4       1 TOP      1364   48.8   2793
##  5       1 UTILITY   125    4.48  2793
##  6       2 BOTTOM      1    3.85    26
##  7       2 JUNGLE     10   38.5     26
##  8       2 MIDDLE      6   23.1     26
##  9       2 TOP         8   30.8     26
## 10       2 UTILITY     1    3.85    26
## # i 14 more rows
```

As we can see from the result, the analysis on the clusters is not useful because of the distribution of nodes in clusters. I tried both with cosine similarity and pearson, and the result was very similar. I decided to keep the pearson similarity because when I used the cosine one, the distribution of nodes in the clusters was even worse (the second cluster for number of nodes had 15 nodes). The only cluster with statistical meaning is the first one (the others have very few nodes to analyze them). It is populated mostly by Toplane nodes, but the fact that the distribution is equal doesn't allow me to feel confident in stating that nodes of the same role form clusters as I supposed before of the analysis.

Conclusion

In conclusion we can say that all my starting questions found an answer, but not the one that I expected. It came out that Junglers do have a high connectivity, but Supports have a higher one, and thinking about it it makes sense because those who know how to play very well (my sample of players) play a lot more around the map than players of lower ranks. On the other hand we have that Toplaners aren't the ones with lower interaction, probably because they usually play champions that make plays and start fights damaging almost everyone in the opponent team getting a lot of assist. Probably this is the same reason why they have the

most interaction with every other role. Instead ADCarries play champions that deal a lot of damages but, usually, they target only few opponent champions because they do not have a lot of area damaging attacks. So ADCarries damages less champions getting less assists. Moreover ADCarries tend to be champions that gain power in the latest part of the game, so even if they play the majority of time in the same lane of Supports it is rare to make plays and take kills only with Supports.